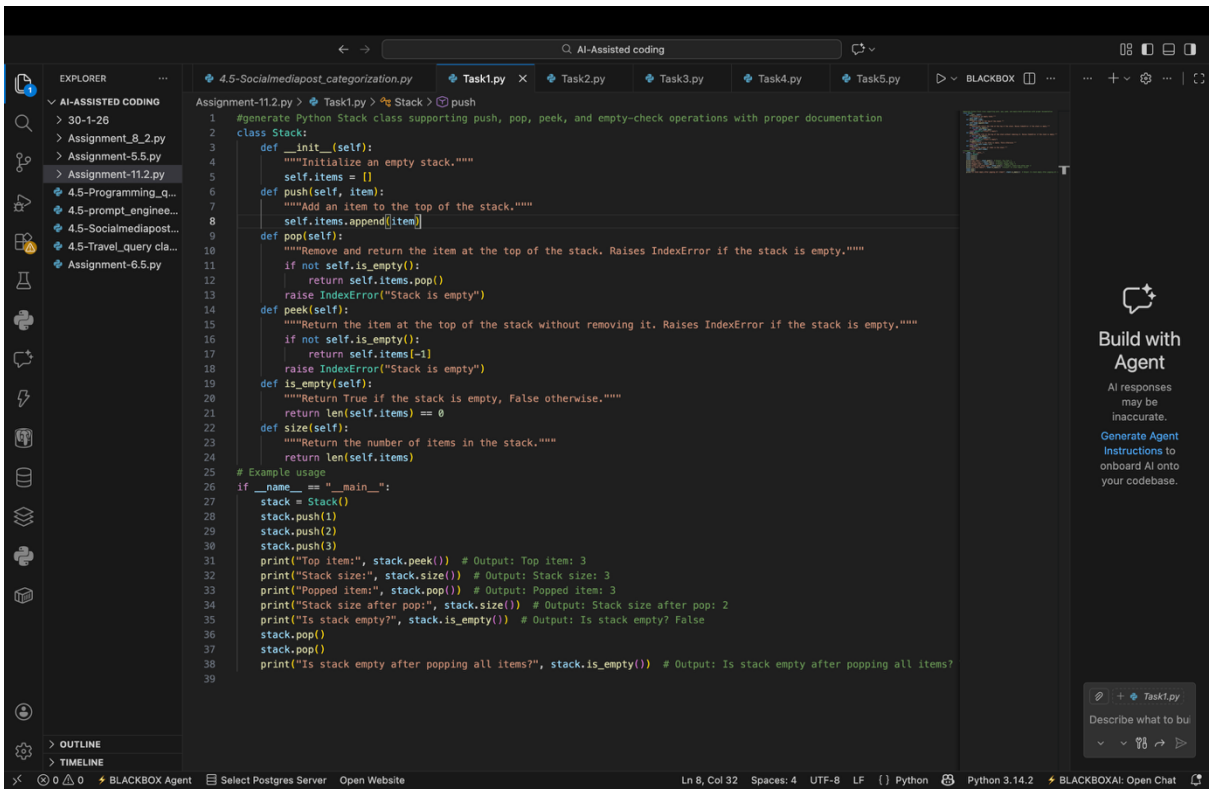
ASSIGNMENT-11.2

Task Description -1:

Prompt:

```
#generate Python Stack class supporting push, pop, peek, and empty-check operations with proper documentation
```

Code:



Output:

```
/usr/local/bin/python3 "/Users/sharanyareddy/Desktop/AI-Assisted coding/Assignment-11.2.py/Task1.py"
sharanyareddy@Sharanyas-MacBook-Air AI-Assisted coding % /usr/local/bin/python3 "/Users/sharanyareddy/Desktop/AI-Assisted coding/Assignment-11.2.py/Task1.py"
Top item: 3
Stack size: 3
Popped item: 3
Stack size after pop: 2
Is stack empty? False
Is stack empty after popping all items? True
sharanyareddy@Sharanyas-MacBook-Air AI-Assisted coding %
```

Justification:

The program implements a stack data structure using a Python list. It supports basic operations such as push, pop, peek,

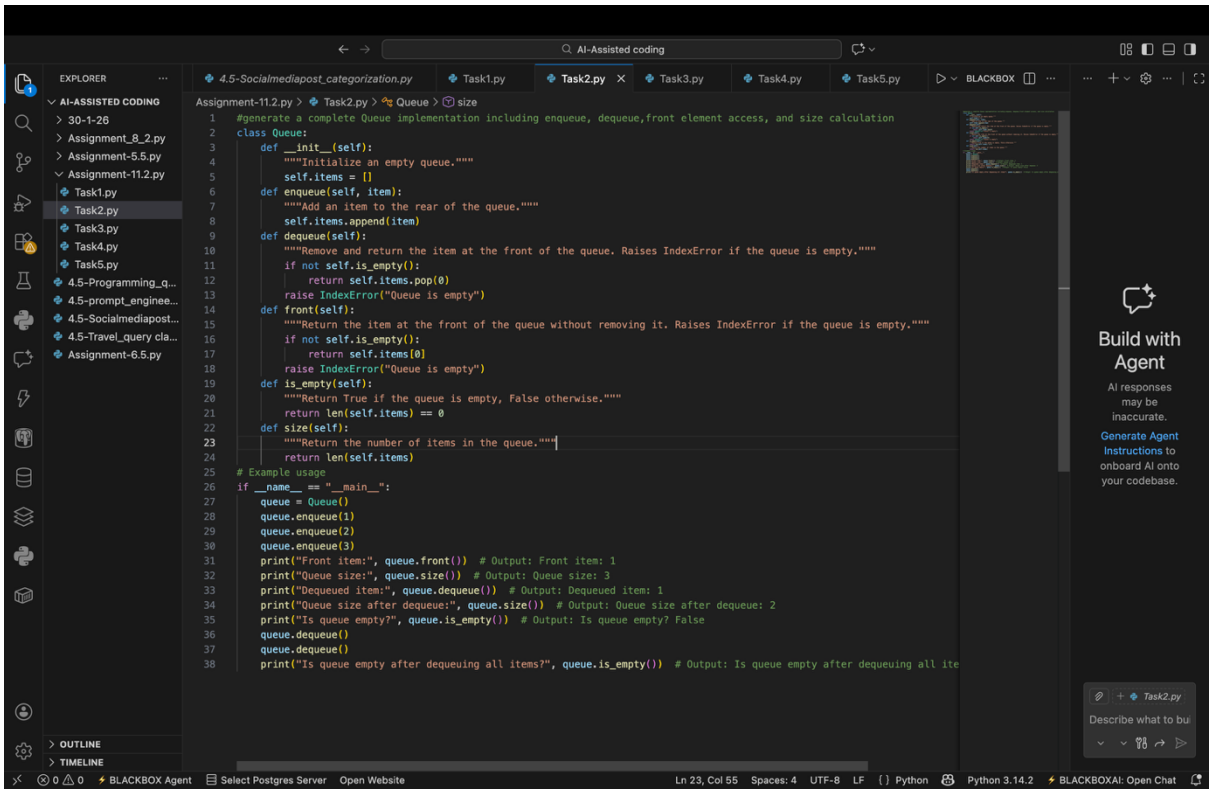
checking if the stack is empty, and finding the size, following the LIFO (Last In, First Out) principle.

Task Description -2:

Prompt:

```
#generate a complete Queue implementation including
enqueue, dequeue,front element access, and size
calculation
```

Code:



Output:

```
Front item: 1
Queue size: 3
Dequeued item: 1
Queue size after dequeue: 2
Is queue empty? False
Is queue empty after dequeuing all items? True
```

Justification: The program implements a Queue data structure using a Python list with operations like enqueue,

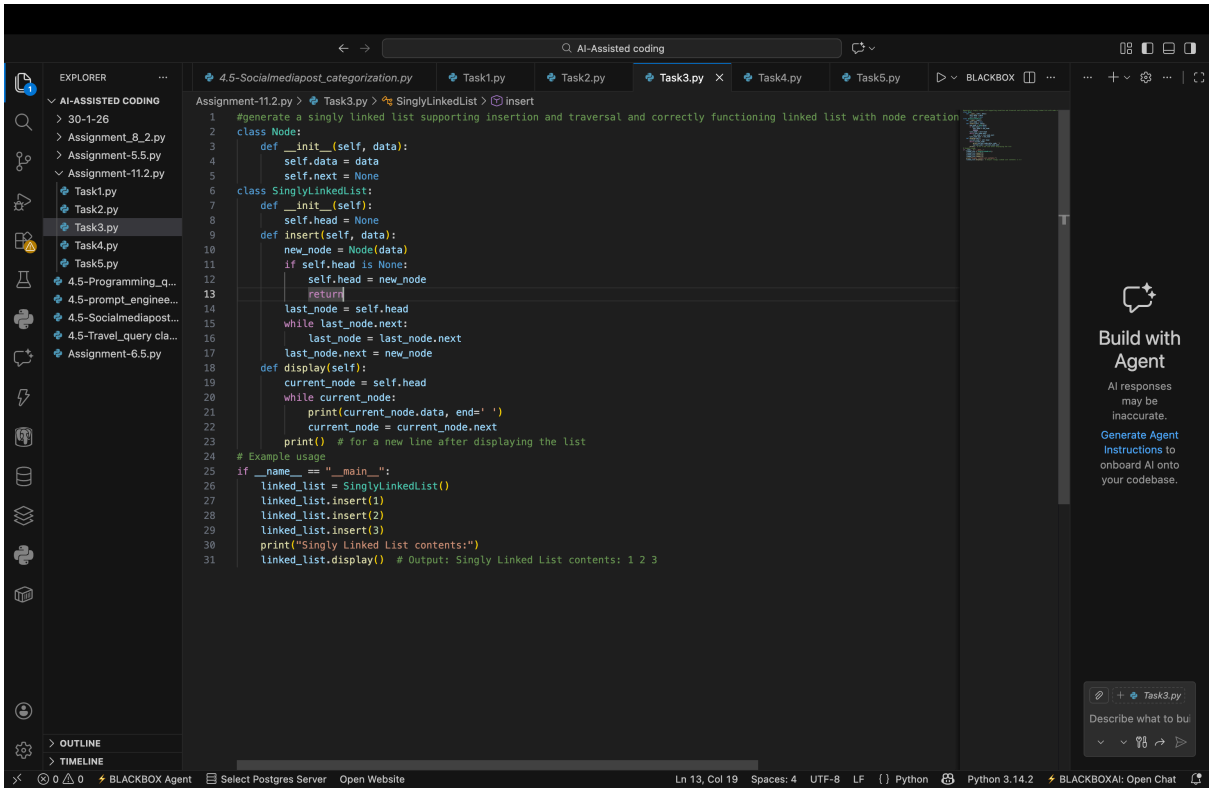
dequeue, peek, checking if empty, and finding the size. It follows the FIFO (First In, First Out) principle, where the first element added to the queue is the first one to be removed.

Task Description -3:

Prompt:

```
#generate a singly linked list supporting insertion and
traversal and correctly functioning linked list with node
creation, insertion logic, and display functionality
```

Code:



Output:

```
Singly Linked List contents:
1 2 3
```

Justification: The program creates a singly linked list where each node stores data and a reference to the next node. It allows

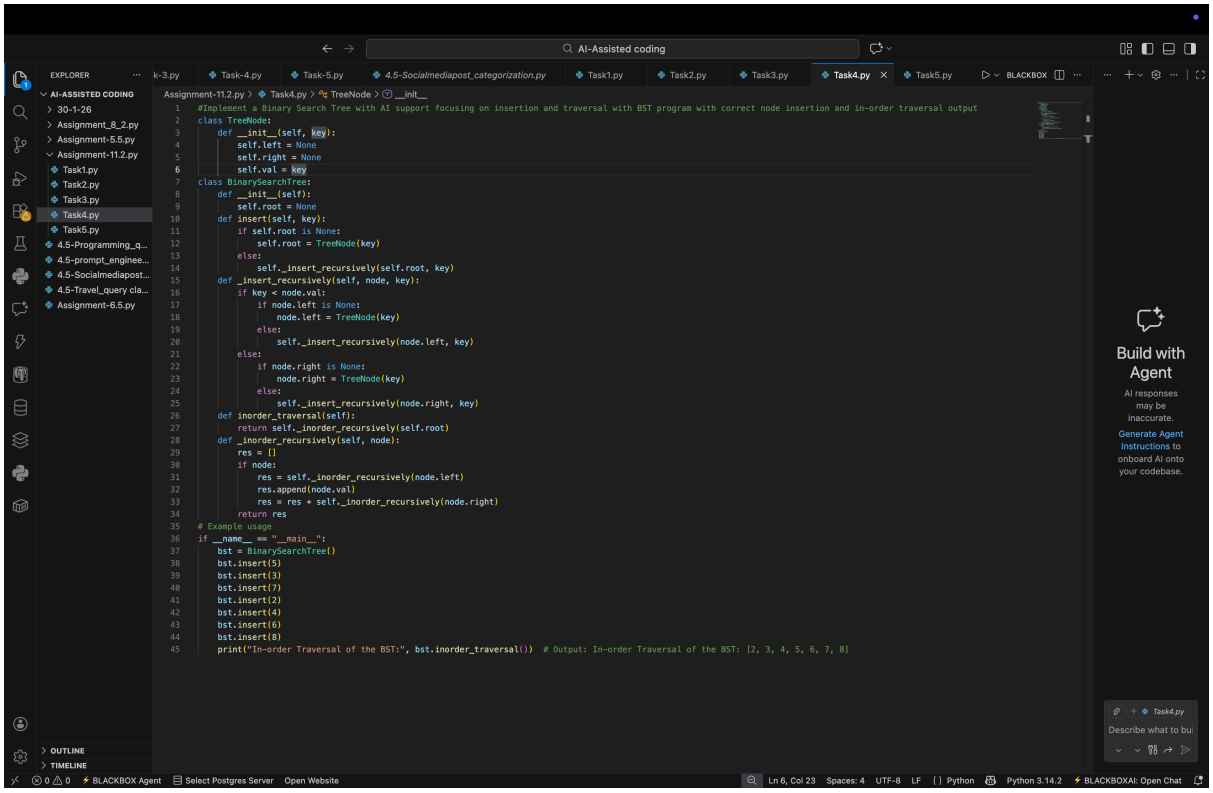
inserting elements and traversing the list to display all node values sequentially.

Task Description -4:

Prompt:

```
#Implement a Binary Search Tree with AI support focusing on
insertion and traversal with BST program with correct node
insertion and in-order traversal output
```

Code:



Output:

```
In-order Traversal of the BST: [2, 3, 4, 5, 6, 7, 8]
```

Justification:

The program implements a Binary Search Tree (BST) where elements are inserted based on their values, with smaller values

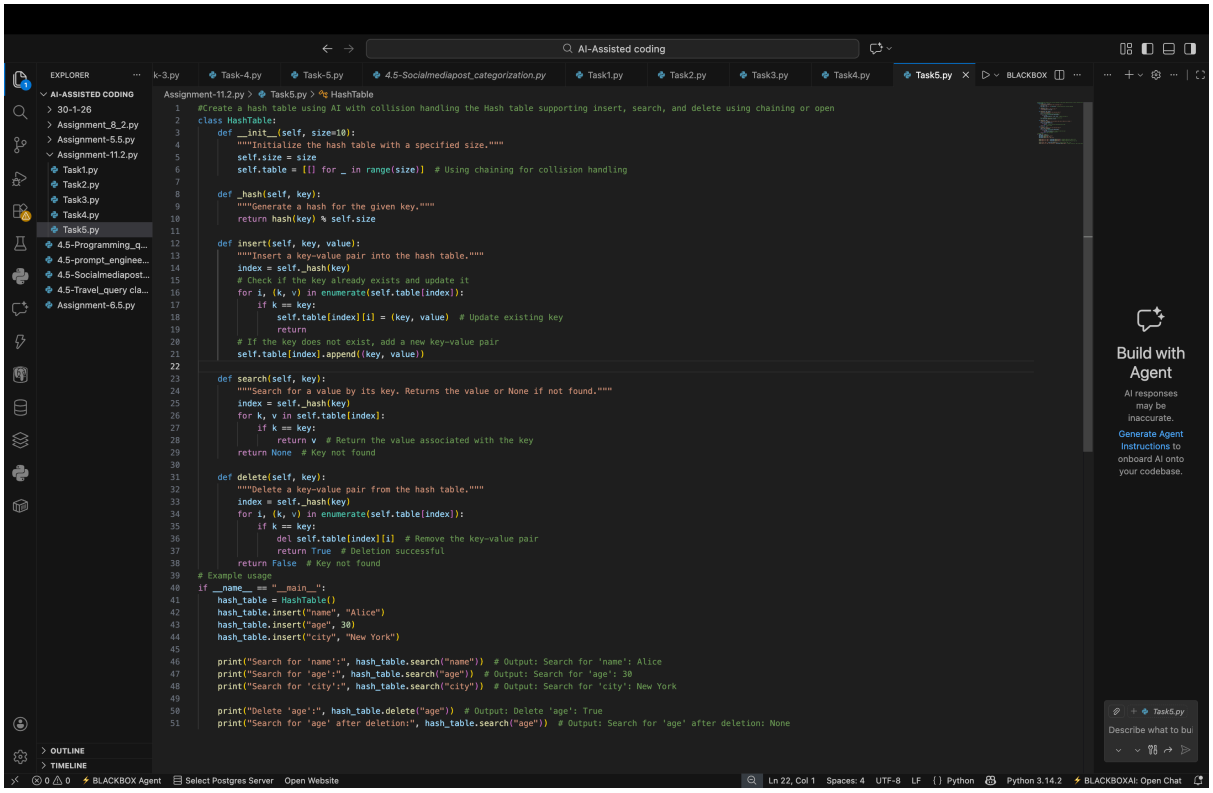
on the left and larger values on the right. The inorder traversal method displays the elements in sorted order.

Task Description -5:

Prompt:

```
#Create a hash table using AI with collision handling the Hash table supporting insert, search, and delete using chaining or open
```

Code:



Output:

```
Search for 'name': Alice
Search for 'age': 30
Search for 'city': New York
Delete 'age': True
Search for 'age' after deletion: None
```

Justification: The program implements a hash table to store key–value pairs using a hash function to determine the index. It

also handles collisions using chaining (list of pairs) and supports operations like insert, get, and delete.